

Refactoring functional programs: past and future

Simon Thompson, University of Kent, UK

Outline

Refactoring ... and functional programming.

Some tools that we have built ... Huiqing Li.

Examples, examples, examples.

Agenda for the future ...

Introduction

Refactoring

Change **how** a program works ...
... without changing **what** it does.

Why refactor?

Extension and reuse

```
loop_a() ->
  receive
    stop -> ok;
    {msg, _Msg, 0} -> loop_a();
    {msg, Msg, N} ->
      io:format("ping!~n"),
      timer:sleep(500),
      b ! {msg, Msg, N - 1},
      loop_a()
  end.
```

Let's turn this into a function

Why refactor?

Extension and reuse

```
loop_a() ->
  receive
    stop -> ok;
    {msg, _Msg, 0} -> loop_a();
    {msg, Msg, N} ->
      io:format("ping!~n"),
      timer:sleep(500),
      b ! {msg, Msg, N - 1},
      loop_a()
  end.
```

```
loop_a() ->
  receive
    stop -> ok;
    {msg, _Msg, 0} -> loop_a();
    {msg, Msg, N} ->
      body(Msg,N),
      loop_a()
  end.
```

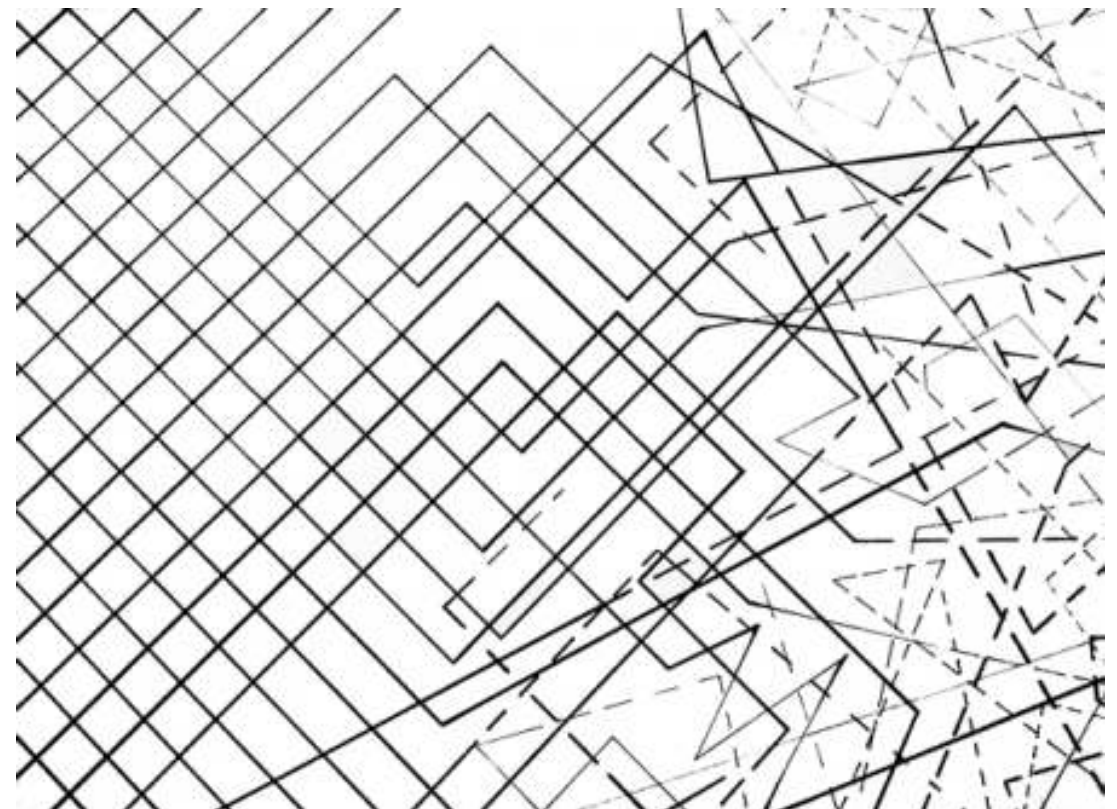
```
body(Msg,N) ->
  io:format("ping!~n"),
  timer:sleep(500),
  b ! {msg, Msg, N - 1}.
```

Why refactor?

Counteract decay ... comprehension

“Clones considered harmful”: detect and eliminate duplicate code.

Improve the module structure: remove loops, for example.



Refactoring *functional* programs

functional programming = expressions + definitions + data types

Computation by **expression** evaluation.

Expressions refer to **definitions** of functions.

Results are values belonging to **data types**.

refactorings = transformations + preconditions + bad smells

A refactoring is a program **transformation**,

whose correctness is guaranteed by validating a **precondition**.

The need for a refactoring is indicated by a **bad smell**.

Classifying refactorings

	local transformation	global transformation	precondition	bad smell
expression	✓		✓	✓
definition	✓	✓	✓	✓
data type		✓	✓	

Classifying refactorings

	local transformation	global transformation	precondition	bad smell
expression	✓		✓	✓
definition	✓	✓	✓	✓
data type		✓	✓	

Tidier for Erlang
HLint for Haskell

Refactoring *functional* programs

Highly expressive expression language ... Tidier, HLint,

More abstractions available:

- can wrap side-effecting code in a closure;

- can abstract over functionality, and not just data.

Semantics “cleaner” even if not fully formal.

Potentially more trustworthy:

- semantics and implementation language.

How to refactor?

By hand ... using an editor.

Flexible ... but error-prone.

Infeasible in the large.

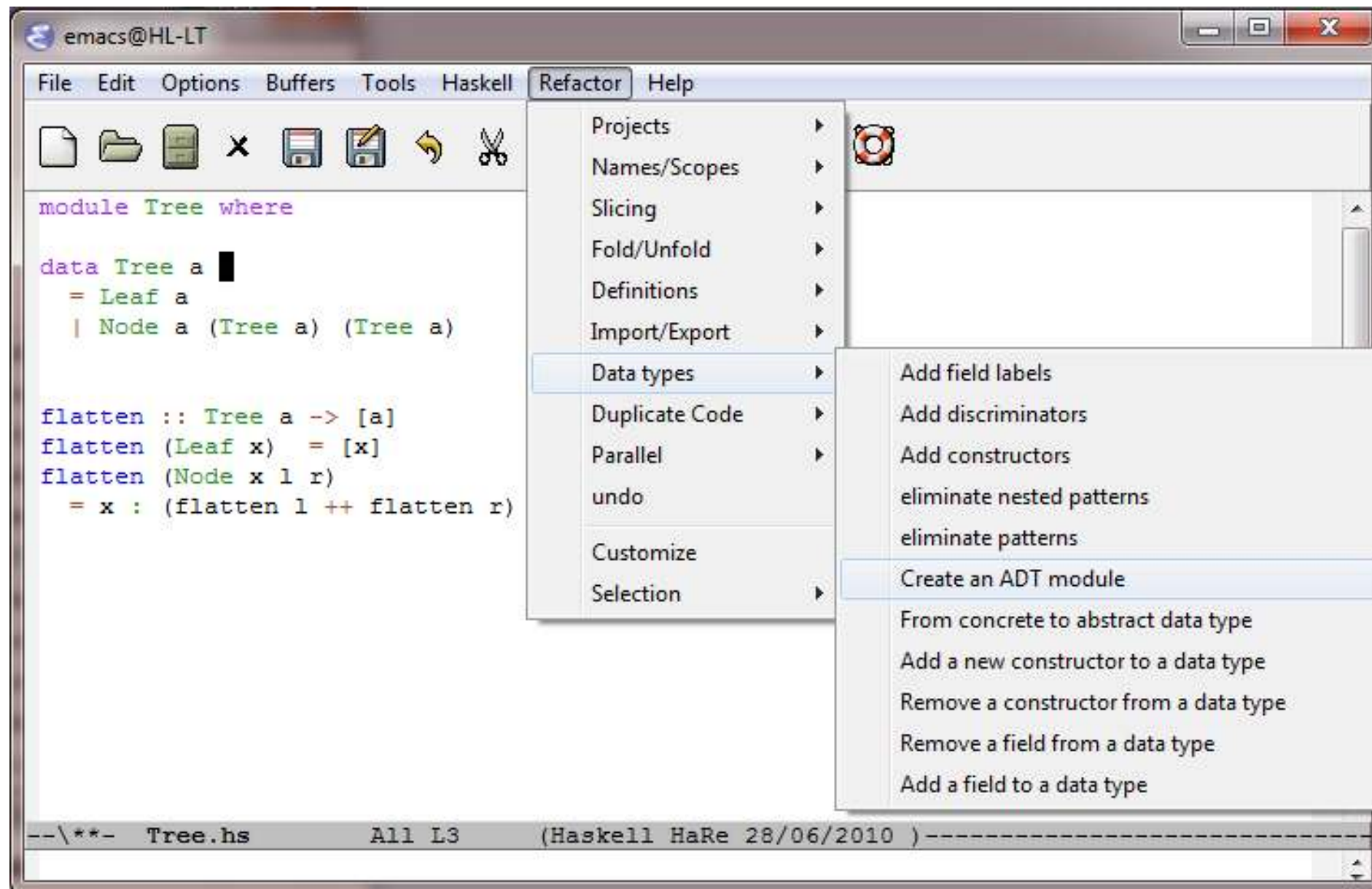
Tool-supported.

Handle atoms, types, names, side-effects, ...

Scalable to large-code bases: module-aware.

Integrated with tests, macros, ...

HaRe



Wrangler

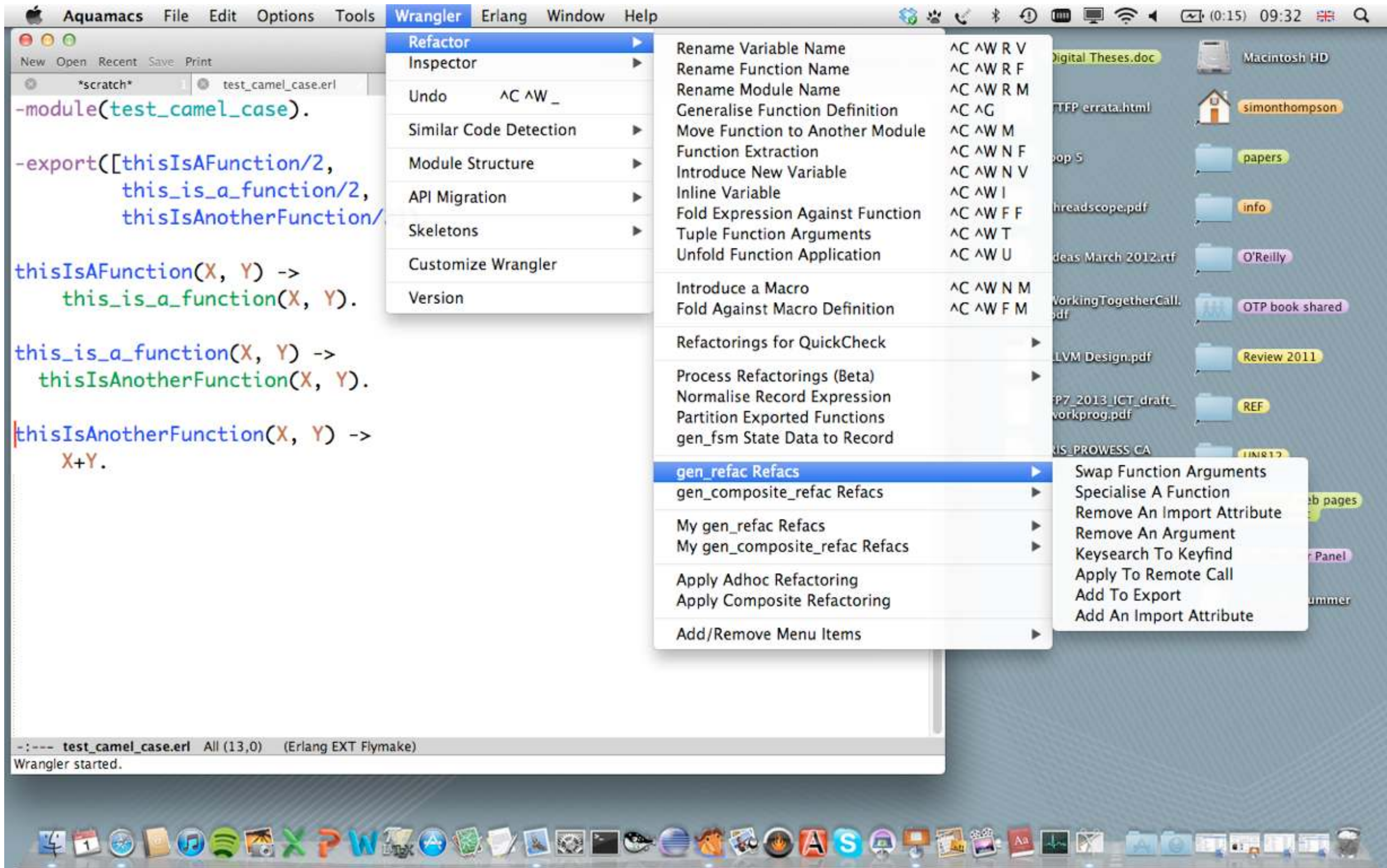
Clone detection
and removal

Module structure
improvement

DSL for composite
refactorings

API: define new
refactorings

Basic refactorings: structural, macro,
process and test-framework related



Wrangler in a nutshell

Automate the simple things, and ...

... provide decision support tools otherwise.

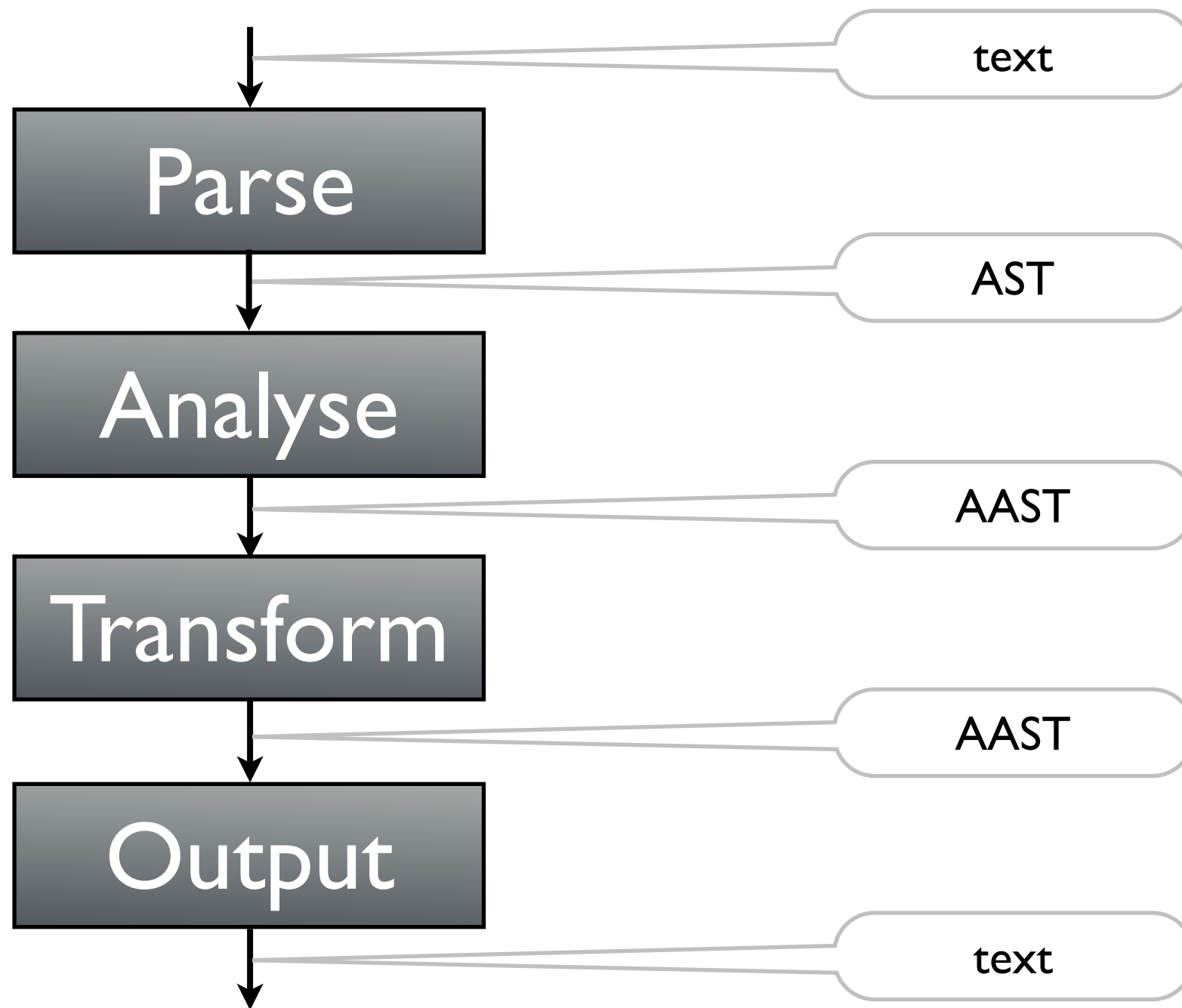
Embed in common IDEs: emacs, eclipse, ...

Handle full language, multiple modules, tests, ...

Faithful to layout and comments.

Build in Erlang and apply the tool to itself.

Under the hood



Examples

Examples

Basic refactorings ...

... and some of their complexity.

Helping the user ...

... clone detection, module structure.

Working in specialised domains ...

... web services, testing frameworks.

Extensibility ...

... an API and a DSL.

Getting started ... what did we mean?

Generalisation ... in Haskell

```
f x y z = length ((2:x) ++ []) +  
           length (True:y) ++ [] +  
           length (3:z) ++ []
```

Generalise over the `[]`.

What do you mean: one, all, some?

```
f x y z w = length ((2:x) ++ w) +  
             length (True:y) ++ w +  
             length (3:z) ++ []
```

What is the type of `w`?

Generalisation ... in Erlang

`f([]) -> ok;`

Call `f([2,2,3])`

`f([X|Xs]) ->
 io:format("~p~t", [X]),
 f(Xs).`

Generalise over `io:format("~p~t", [X]).`

Generalisation ... in Erlang

```
f([]) -> ok;
```

Call `f([2,2,3])`

```
f([X|Xs]) ->  
    io:format("~p~t", [X]),  
    f(Xs).
```

Generalise over `io:format("~p~t", [X])`.

What about the side-effect, the free variable?

```
f([],h) -> ok;
```

Call `f([2,2,3],
 fun(X) -> io:format("~p~t", [X]) end)`

```
f([X|Xs],h) ->  
    h(X),  
    f(Xs).
```


Lifting definitions ... in Haskell

```
h x = x + g x
  where
    = x + con
    con = 37
```

`g x`

Lift `g` to be a top-level definition. What about `con`?

Lambda lift?

```
h x = x + g con x
  where
    = 37
```

`con`

```
g con x = x + con
```


Lifting definitions ... in Haskell

```
h x = x + g x
  where
    = x + con
    con = 37
```

`g x`

Lift `g` to be a top-level definition. What about `con`?

Localise before lifting?

```
h x = x + g x

g x = x + con
  where
    = 37
```

`con`

Lifting definitions ... in Haskell

```
h x = x + g x
  where
    = x + con
    con = 37
```

`g x`

Lift `g` to be a top-level definition. What about `con`?

Lift all dependents?

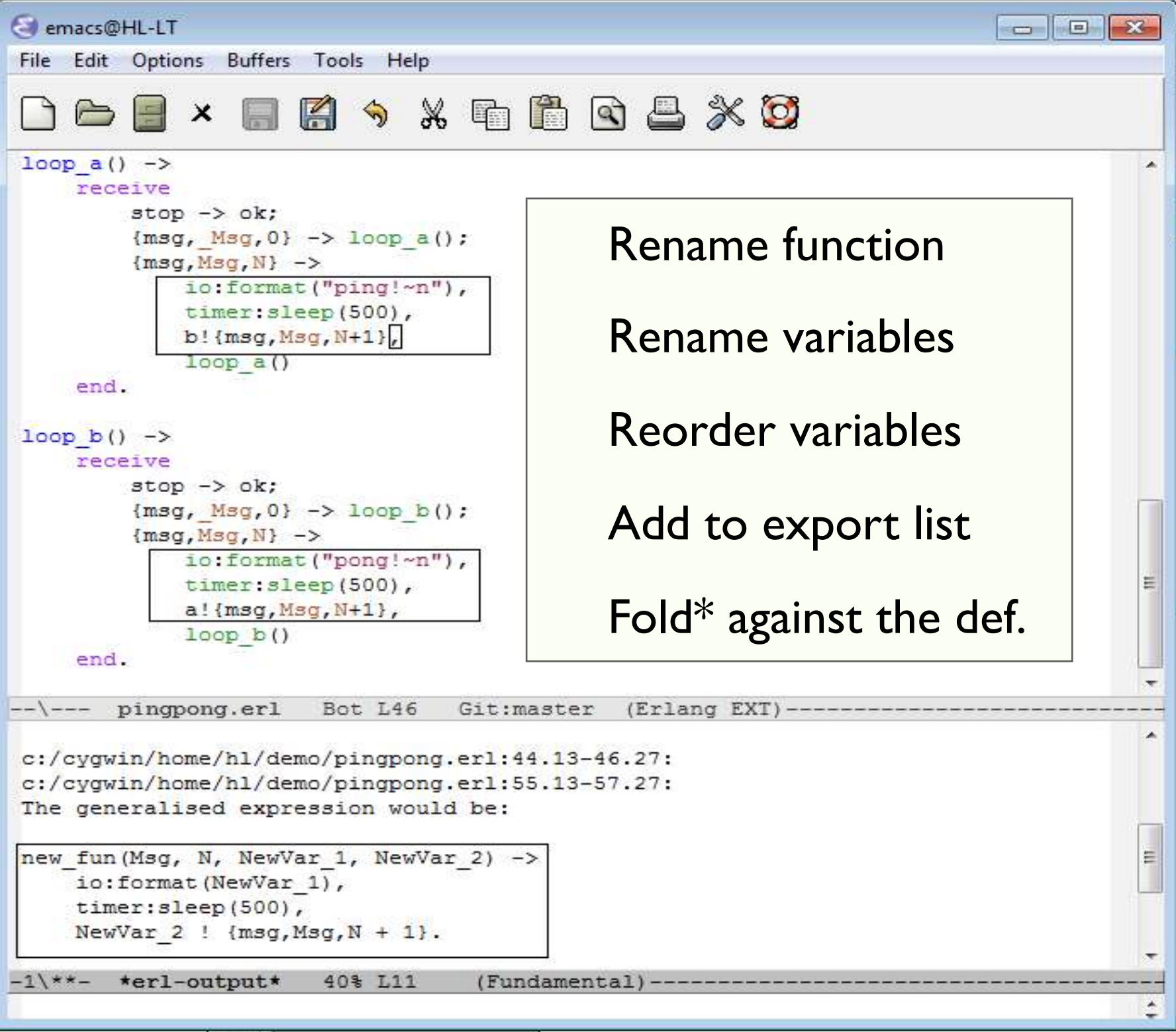
```
h x = x + g x

g x = x + con

con = 37
```

Being informed ... in particular domains

Clone removal



emacs@HL-LT

File Edit Options Buffers Tools Help

loop_a() ->
receive
stop -> ok;
{msg, _Msg, 0} -> loop_a();
{msg, Msg, N} ->
io:format("ping!~n"),
timer:sleep(500),
b!{msg,Msg,N+1},
loop_a()
end.

loop_b() ->
receive
stop -> ok;
{msg, _Msg, 0} -> loop_b();
{msg, Msg, N} ->
io:format("pong!~n"),
timer:sleep(500),
a!{msg,Msg,N+1},
loop_b()
end.

--\--- pingpong.erl Bot L46 Git:master (Erlang EXT)-----

c:/cygwin/home/hl/demo/pingpong.erl:44.13-46.27:
c:/cygwin/home/hl/demo/pingpong.erl:55.13-57.27:
The generalised expression would be:

new_fun(Msg, N, NewVar_1, NewVar_2) ->
io:format(NewVar_1),
timer:sleep(500),
NewVar_2 ! {msg,Msg,N + 1}.

-1*- *erl-output* 40% L11 (Fundamental)-----

Rename function
Rename variables
Reorder variables
Add to export list
Fold* against the def.

Web Services Testing

Support property-based testing of web services using QuickCheck and to evolve test code between versions of web services.

From WSDL createof data type definitions and data generators, connector module, and skeleton state machine.

Automatic inference of web service interface changes from WSDL.

A set of domain-specific refactorings, e.g. add operation argument.

Automatic creation of refactoring scripts to make the changes.

Extending it yourself

Extensibility: API + DSL

API

Describe entirely new ‘atomic’ refactorings from scratch.

e.g. swap args, delete argument.

We assume you know Erlang, but not internals of the syntax.

DSL

A language to script composite refactorings on top of simpler ones.

e.g. remove clone, migrate from old to new API.

We *embed* in Erlang, to use the language in the “scripts”.

API: templates and rules ... in Erlang

`?RULE(Template, NewCode, Cond)`

The old code, the new code and the pre-condition.

```
rule({M,F,A}, N) ->
    ?RULE(?T("F@(Args@@)"),
        begin
            NewArgs@@=delete(N, Args@@),
            ?TO_AST("F@(NewArgs@@)")
        end,
        refac_api:fun_define_info(F@) == {M,F,A}).
```

`delete(N, List) -> ... delete Nth elem of List ...`

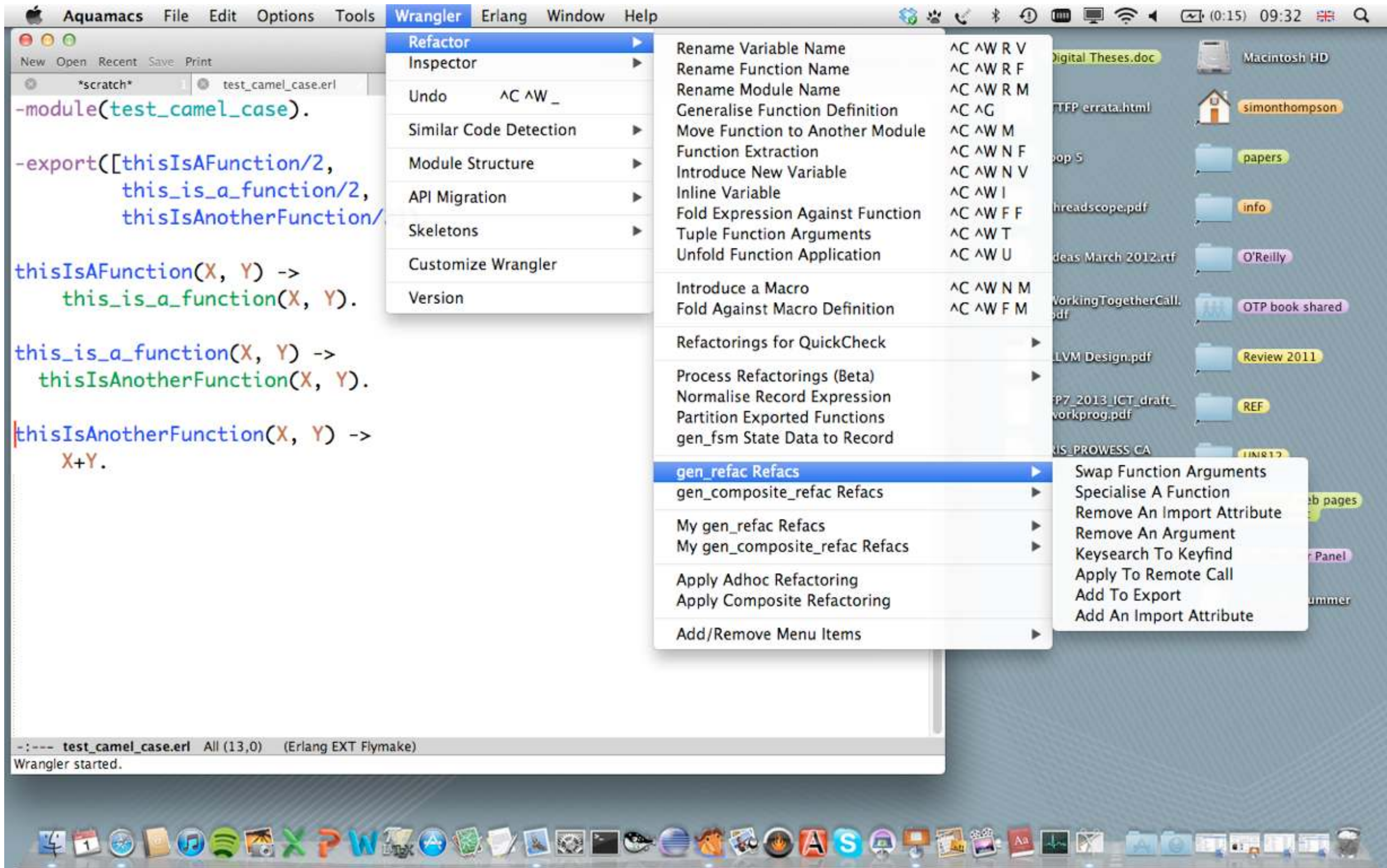
Wrangler API

Context
available for
pre-conditions

Traversals
describe how
rules are applied

Rules describe transformations

Templates describe expressions



DSL ... not just a script

Tracking changing names and positions.

Generating refactoring commands.

Dealing with failure.

User control of execution.

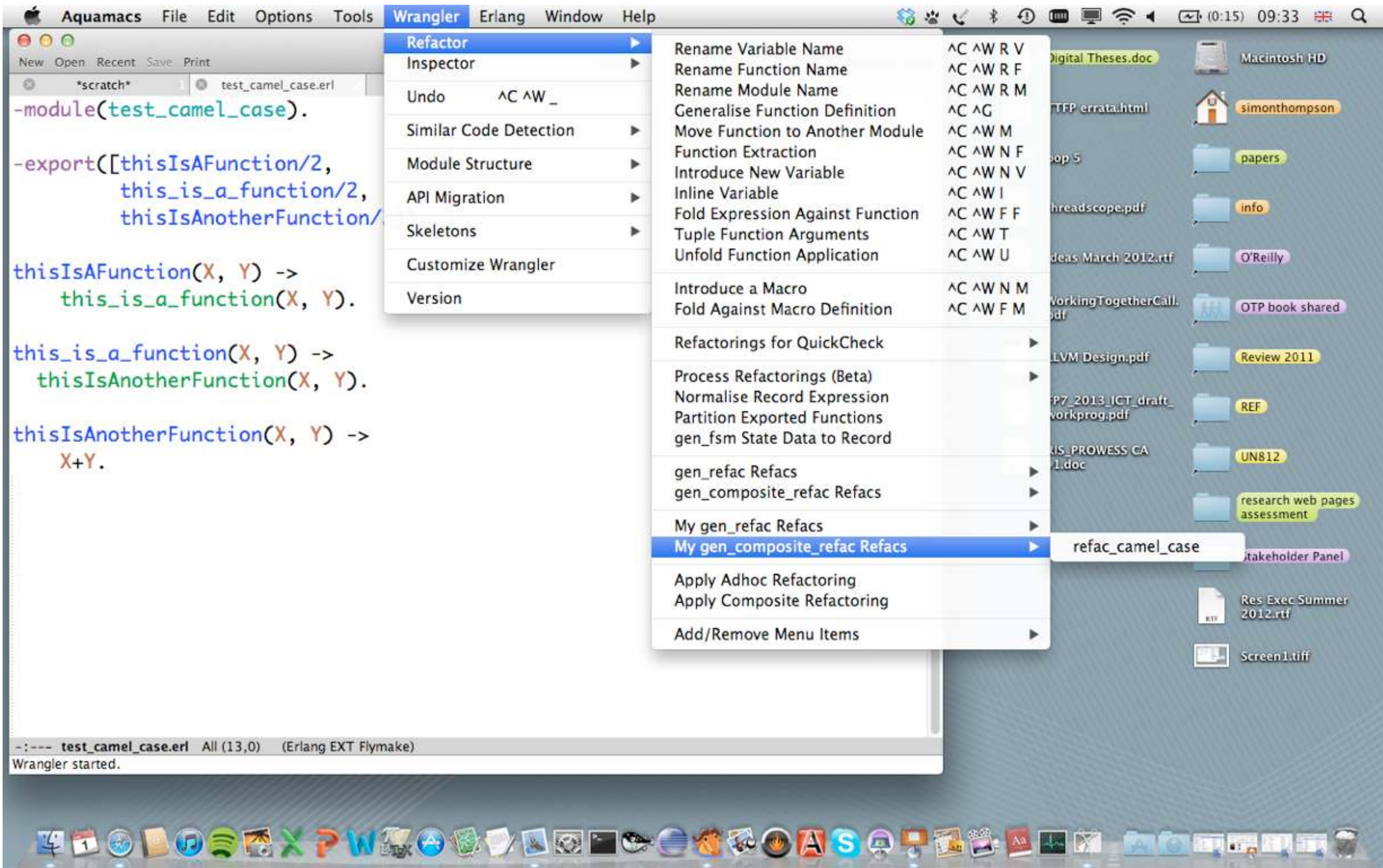
Deals with the pragmatics of composition, rather than the theory.

Generation: camel case

?refac_(CmdName, Args, Scope)

Args: modules, camelCase functions, new names.

```
?refac_(rename_fun,  
  [{file, fun(_File)-> true end},  
    fun({F, _A}) ->  
      camelCase_to_camel_case(F) /= F  
    end,  
    {generator, fun({_File, F, _A}) ->  
      camelCase_to_camel_case(F)  
    end}],  
  SearchPaths).
```

Clone removal in the DSL

Transaction as a whole ... non-transactional components OK.

Not just an API: `?transaction` etc. modify interpretation of what they enclose ...

```
?transaction(  
  [?interactive( RENAME FUNCTION )  
    ?refac_( RENAME ALL VARIABLES OF THE FORM NewVar*)  
    ?repeat_interactive( SWAP ARGUMENTS )  
    ?if_then( EXPORT IF NOT ALREADY )  
    ?non_transaction( FOLD INSTANCES OF THE CLONE )  
  ]).
```

Looking forward

“Why should I trust my code to your tool?”

Benefit $\gg$ risk: removing bug preconditions

Scenario: building Erlang models for C code at Quviq AB.

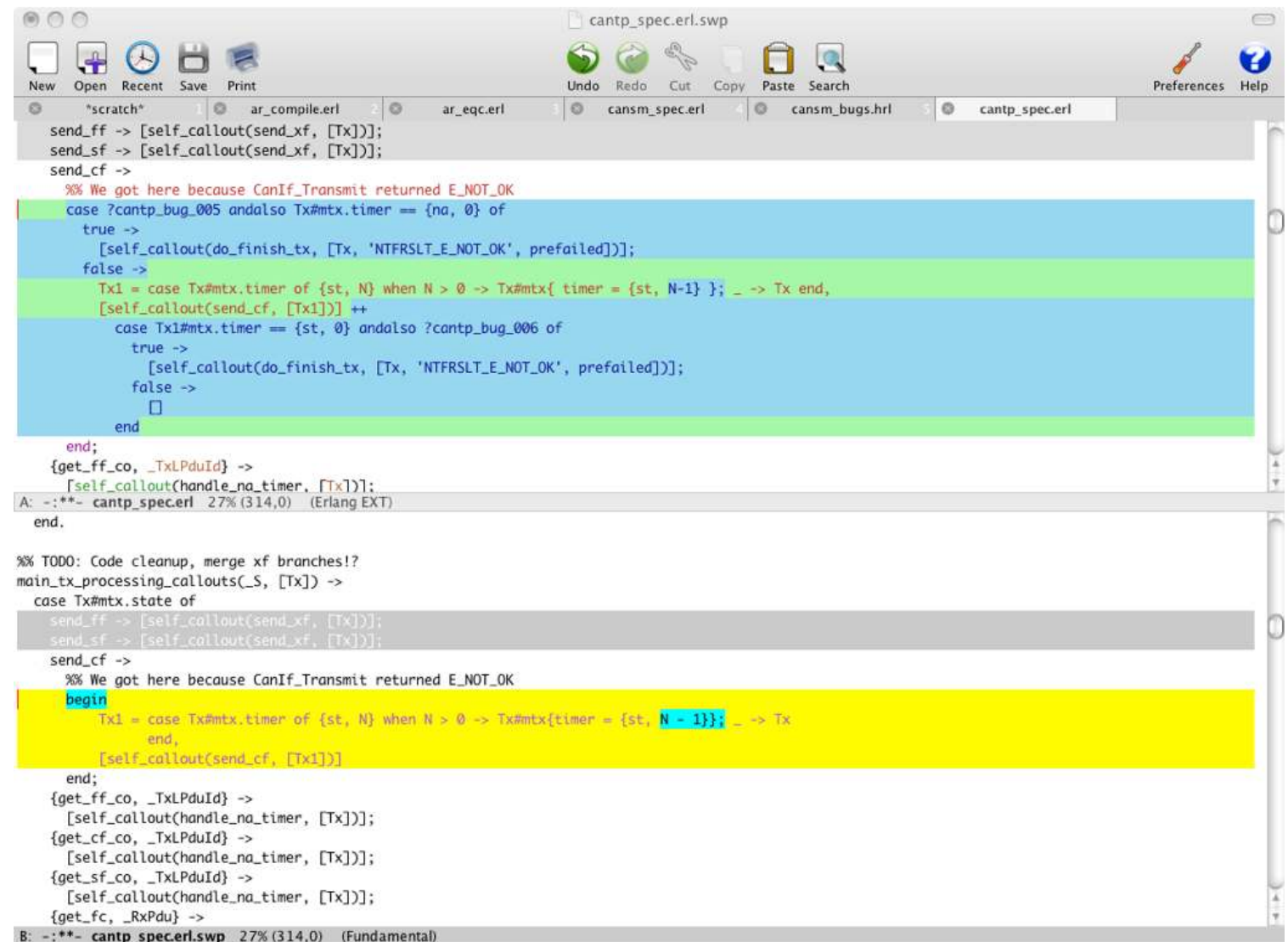
For buggy code, want to avoid hitting the same bugs all the time.

Add bug precondition macros ...

... but want to remove in delivered code.

DSL + API.

And you can see the changes ...



```
cantp_spec.erl.swp
New Open Recent Save Print Undo Redo Cut Copy Paste Search Preferences Help

"scratch" ar_compile.erl ar_eqc.erl cansm_spec.erl cansm_bugs.hrl cantp_spec.erl

send_ff -> [self_callout(send_xf, [Tx])];
send_sf -> [self_callout(send_xf, [Tx])];
send_cf ->
    %% We got here because CanIf_Transmit returned E_NOT_OK
    case ?cantp_bug_005 andalso Tx#mtx.timer == {na, 0} of
    true ->
        [self_callout(do_finish_tx, [Tx, 'NTRSLT_E_NOT_OK', prefailed])];
    false ->
        Tx1 = case Tx#mtx.timer of {st, N} when N > 0 -> Tx#mtx{ timer = {st, N-1} }; _ -> Tx end,
        [self_callout(send_cf, [Tx1])] ++
        case Tx1#mtx.timer == {st, 0} andalso ?cantp_bug_006 of
        true ->
            [self_callout(do_finish_tx, [Tx, 'NTRSLT_E_NOT_OK', prefailed])];
        false ->
            []
        end
    end
end;

{get_ff_co, _TxLPduId} ->
    [self_callout(handle_na_timer, [Tx])];
A: -:** cantp_spec.erl 27% (314,0) (Erlang EXT)
end.

%% TODO: Code cleanup, merge xf branches!?
main_tx_processing_callouts(_S, [Tx]) ->
    case Tx#mtx.state of
    send_ff -> [self_callout(send_xf, [Tx])];
    send_sf -> [self_callout(send_xf, [Tx])];
    send_cf ->
        %% We got here because CanIf_Transmit returned E_NOT_OK
        begin
            Tx1 = case Tx#mtx.timer of {st, N} when N > 0 -> Tx#mtx{timer = {st, N - 1}}; _ -> Tx
            end,
            [self_callout(send_cf, [Tx1])]
        end;
    end;
    {get_ff_co, _TxLPduId} ->
        [self_callout(handle_na_timer, [Tx])];
    {get_cf_co, _TxLPduId} ->
        [self_callout(handle_na_timer, [Tx])];
    {get_sf_co, _TxLPduId} ->
        [self_callout(handle_na_timer, [Tx])];
    {get_fc, _RxPdu} ->
        []
    end;
B: -:** cantp_spec.erl.swp 27% (314,0) (Fundamental)
```

The appearance hasn't changed

```
my_list() ->  
  [ foo,  
    bar,  
    baz,  
    wombat  
  ]
```

{v1, v2, v3}

{v1,v2,v3}

```
data MyType = Foo |  
             Bar |  
             Baz
```

```
my_funny_list() ->  
  [ foo  
    ,bar  
    ,baz  
    ,wombat  
  ]
```

f (g x y)

f \$ g x y

```
data HerType = Foo  
              | Bar  
              | Baz
```

Preserving meaning

What are we preserving?

Where are we preserving it?

Individual results or the refactoring tool itself?

Equivalences

Testing equivalence: $\forall$ test data [finite]

PBT equivalence: $\forall$ random test data [finite, but unbounded]

Extensional equivalence: $\forall$ input data [infinite]

(Annotated) abstract syntax tree (with some quotient?)

Textual

Question: varieties of $\downarrow$: may be happy to converge on *more* inputs?

tool or results

test or verify

tool or results

test or verify

tool or results

X	

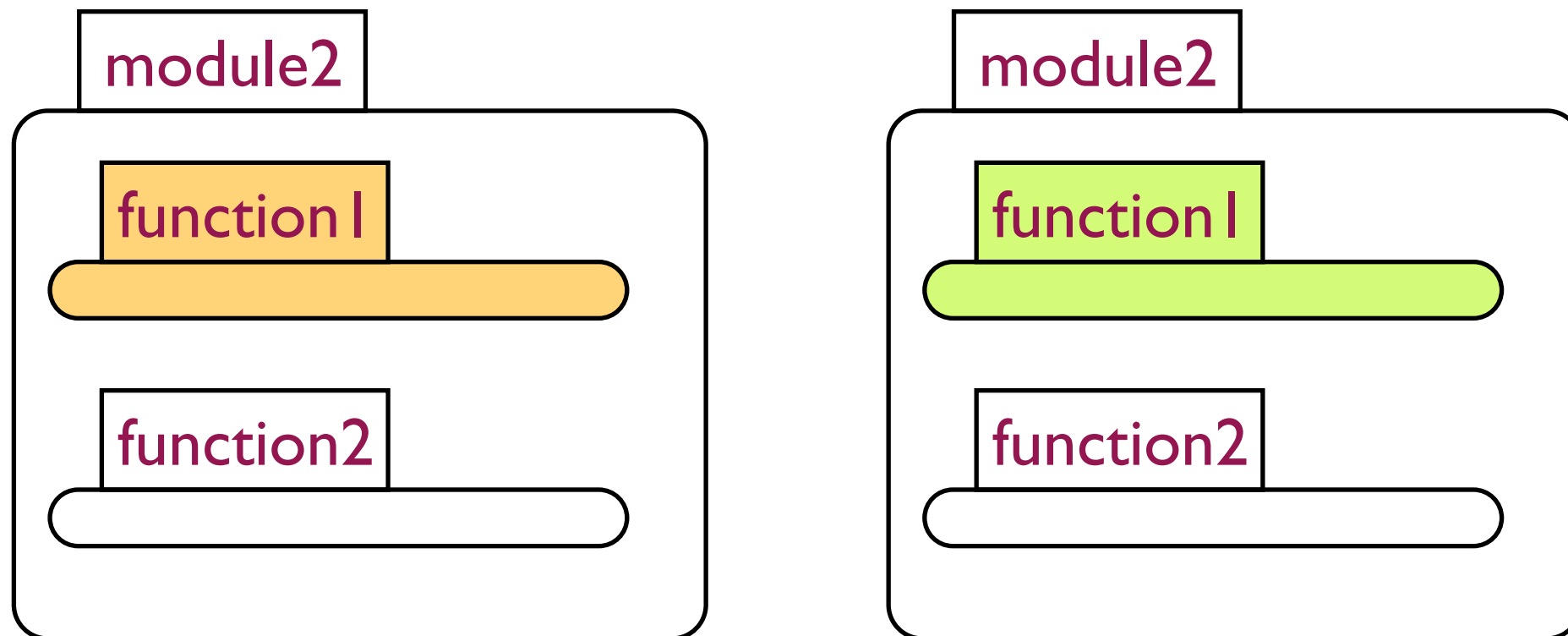
test or verify

Testing two refactoring tools

Compare the results of **tool1** and **tool2** ...

... either by testing both, or directly comparing the code / ASTs.

Similar to compiler comparisons and Eclipse vs NetBeans (*Dig et al*).

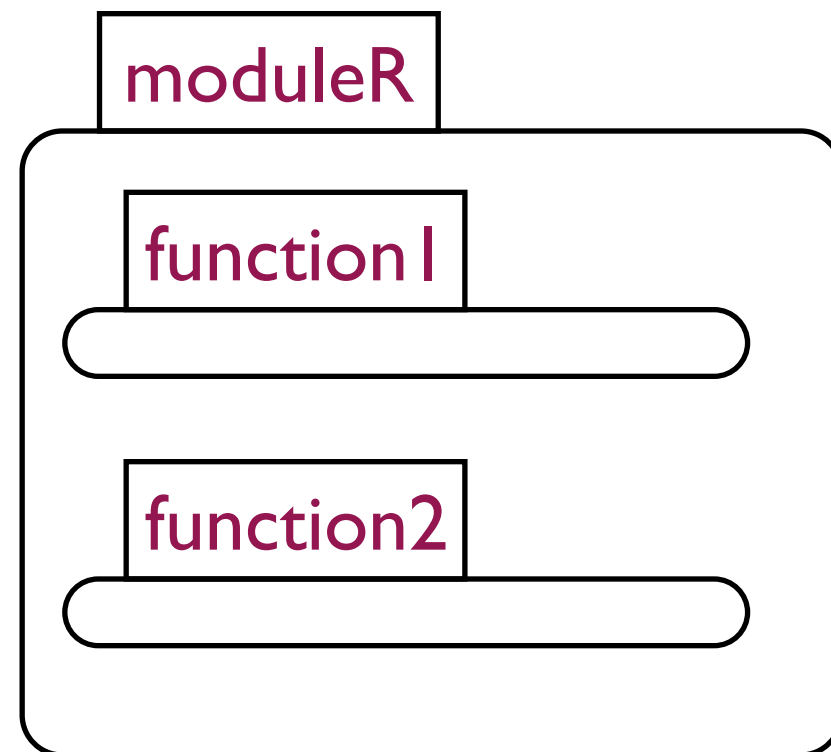
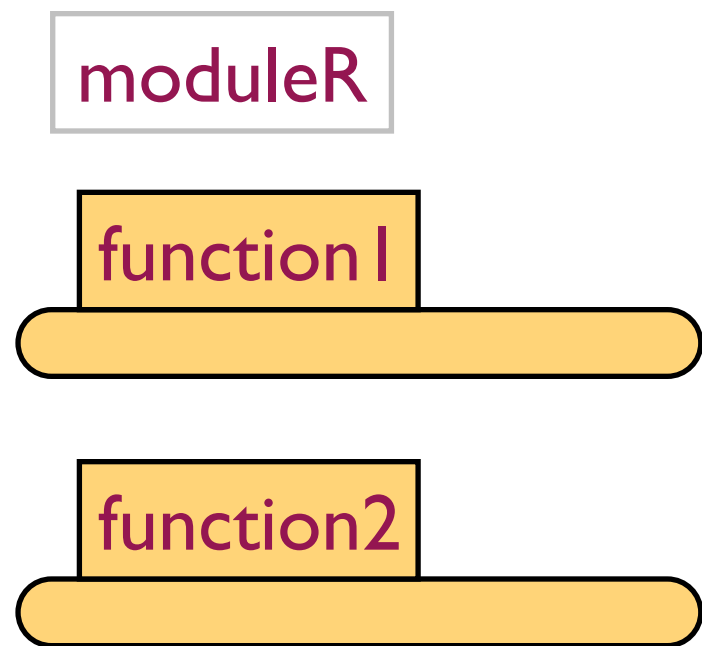


Fully random

Generate random modules,

... generate random refactoring commands,

... and check $\equiv$ with random inputs. (w/ Drienyovszky, Horpácsi).



tool or results

	X

test or verify

Tool verification (with Nik Sultana)

$$\forall p. (Q\ p) \longrightarrow (T\ p) \simeq p$$

Deep embeddings of small languages:

- ... potentially name-capturing λ -calculus

- ... PCF with unit and sum types.

Isabelle/HOL: LCF-style secure proof checking.

Formalisation of meta-theory: variable binding, free / bound variables, capture, fresh variables, typing rules, etc ...

- ... principally to support pre-conditions.

Variable capturing substitution

$\varepsilon[M/x]$	$\stackrel{\text{def}}{=} \varepsilon$
$(y := N)[M/x]$	$\stackrel{\text{def}}{=} \text{if } x = y \text{ then } y := N$ $\text{else } y := (N[M/x])$
$(D_1 \parallel D_2)[M/x]$	$\stackrel{\text{def}}{=} \text{if } x \in DVTopd(D_1 \parallel D_2)$ $\text{then } (D_1 \parallel D_2)$ $\text{else } (D_1[M/x] \parallel D_2[M/x])$
$i[M/x]$	$\stackrel{\text{def}}{=} \text{if } x = i \text{ then } M \text{ else } i$
$(\lambda i.N)[M/x]$	$\stackrel{\text{def}}{=} \text{if } x = i \text{ then } \lambda i.N$ $\text{else } \lambda i.(N[M/x])$
$(N \cdot N')[M/x]$	$\stackrel{\text{def}}{=} (N[M/x]) \cdot (N'[M/x])$
$(\text{letrec } D \text{ in } N)[M/x]$	$\stackrel{\text{def}}{=} \text{if } x \in DVTopd(\text{letrec } D \text{ in } N)$ $\text{then } (\text{letrec } D \text{ in } N)$ $\text{else } \text{letrec } (D[M/x]) \text{ in } (N[M/x])$

tool or results

	X

test or verify

Automatically verify instances of refactorings

Prove the equivalence of the particular pair of functions / systems using an SMT solver ...

... SMT solvers linked to Haskell by `Data.SBV` (Levent Erkok).

Manifestly clear what is being checked.

The approach delegates trust to the SMT solver ...

... can choose other solvers, and examine counter-examples.

Also possible for Erlang using e.g. McErlang model checker.

Example

```
module Before where
```

```
h :: Integer->Integer->Integer
```

```
h x y = g y + f (g y)
```

```
g :: Integer->Integer
```

```
g x = 3*x + f x
```

```
f :: Integer->Integer
```

```
f x = x + 1
```


Example: renaming

```
module Before where
```

```
h :: Integer->Integer->Integer
```

```
h x y = g y + f (g y)
```

```
g :: Integer->Integer
```

```
g x = 3*x + f x
```

```
f :: Integer->Integer
```

```
f x = x + 1
```

```
module After where
```

```
h :: Integer->Integer->Integer
```

```
h x y = k y + f (k y)
```

```
k :: Integer->Integer
```

```
k x = 3*x + f x
```

```
f :: Integer->Integer
```

```
f x = x + 1
```



```
{-# LANGUAGE ScopedTypeVariables #-}
```

```
module RefacProof where
```

```
import Data.SBV
```

```
{-# LANGUAGE ScopedTypeVariables #-}
```

```
module RefacProof where
```

```
import Data.SBV
```

```
h :: Integer->Integer->Integer
```

```
h x y = g y + f (g y)
```

```
g :: Integer->Integer
```

```
g x = 3*x + f x
```

```
{-# LANGUAGE ScopedTypeVariables #-}  
  
module RefacProof where  
  
import Data.SBV
```

```
h :: Integer->Integer->Integer  
h x y = g y + f (g y)  
  
g :: Integer->Integer  
g x = 3*x + f x
```

```
h' :: Integer->Integer->Integer  
h' x y = k y + f (k y)  
  
k :: Integer->Integer  
k x = 3*x + f x
```

```
{-# LANGUAGE ScopedTypeVariables #-}
```

```
module RefacProof where
```

```
import Data.SBV
```

```
h :: Integer->Integer->Integer
```

```
h x y = g y + f (g y)
```

```
g :: Integer->Integer
```

```
g x = 3*x + f x
```

```
h' :: Integer->Integer->Integer
```

```
h' x y = k y + f (k y)
```

```
k :: Integer->Integer
```

```
k x = 3*x + f x
```

```
-- f can be treated as an uninterpreted symbol
```

```
f = uninterpret "f"
```

```
-- Properties
```

```
propertyk = prove $ \ (x::SInteger) -> g x .== k x
```

```
propertyh = prove $ \ (x::SInteger) (y::SInteger) -> h x y .== h' x y
```

```
h :: Integer->Integer->Integer
```

```
h x y = g y + f (g y)
```

```
g :: Integer->Integer
```

```
g x = 3*x + f x
```

```
h' :: Integer->Integer->Integer
```

```
h' x y = k y + f (k y)
```

```
k :: Integer->Integer
```

```
k x = 3*x + f x
```

```
-- f can be treated as an uninterpreted symbol
```

```
f = uninterpret "f"
```

```
-- Properties
```

```
propertyk = prove $ \ (x::SInteger) -> g x .== k x
```

```
propertyh = prove $ \ (x::SInteger) (y::SInteger) -> h x y .== h' x y
```

```
*Refac2> propertyk
```

```
Q.E.D.
```

```
*Refac2> propertyh
```

```
Q.E.D.
```

```
h :: Integer->Integer->Integer
```

```
h x y = g y + f (g y)
```

```
  where
```

```
    g z = z*z
```

```
g :: Integer->Integer
```

```
g x = 3*x + f x
```

```
h :: Integer->Integer->Integer
```

```
h x y = g y + f (g y)
  where
    g z = z*z
```

```
g :: Integer->Integer
```

```
g x = 3*x + f x
```

```
h' :: Integer->Integer->Integer
```

```
h' x y = k y + f (k y)
  where
    g z = z*z
```

```
k :: Integer->Integer
```

```
k x = 3*x + f x
```



```
h :: Integer->Integer->Integer
```

```
h x y = g y + f (g y)
  where
    g z = z*z
```

```
g :: Integer->Integer
```

```
g x = 3*x + f x
```

```
h' :: Integer->Integer->Integer
```

```
h' x y = k y + f (k y)
  where
    g z = z*z
```

```
k :: Integer->Integer
```

```
k x = 3*x + f x
```

```
f = uninterpret "f"
```

```
propertyk = prove $ \ (x::SInteger) -> g x .== k x
```

```
propertyh = prove $ \ (x::SInteger) (y::SInteger) -> h x y .== h' x y
```



```
h :: Integer->Integer->Integer
```

```
h x y = g y + f (g y)
  where
    g z = z*z
```

```
g :: Integer->Integer
```

```
g x = 3*x + f x
```

```
h' :: Integer->Integer->Integer
```

```
h' x y = k y + f (k y)
  where
    g z = z*z
```

```
k :: Integer->Integer
```

```
k x = 3*x + f x
```

```
f = uninterpret "f"
```

```
propertyk = prove $ \ (x::SInteger) -> g x .== k x
```

```
propertyh = prove $ \ (x::SInteger) (y::SInteger) -> h x y .== h' x y
```

```
*Refac2> propertyk
```

```
Q.E.D.
```

```
*Refac2> propertyh
```

```
Falsifiable. Counter-example:
```

```
  s0 = 0 :: SInteger
```

```
  s1 = -1 :: SInteger
```

“How do I refactor my data representation?”

Changing data representation

Modify the implementation of a particular type (synonym).

But don't modify all occurrences of `(Int,Bool)`, ... scope issue.

```
type Rep = (Int,Bool)
```

```
f :: Rep -> Int  
f (n,_) = n + 42
```

```
g :: (Int,Bool) -> Rep  
g (n,b) =  
  if b then (n,b) else (-n,b)
```

```
h :: Rep -> Bool  
h = snd
```

```
type Rep = (Bool,Int)
```

```
f :: Rep -> Int  
f (_,n) = n + 42
```

```
g :: (Int,Bool) -> Rep  
g (n,b) =  
  if b then (b,n) else (b,-n)
```

```
h :: Rep -> Bool  
h = snd . flip
```

Changing data representations

Where does it get interesting?

Introducing monad or applicative, e.g.

`Int -> (x,Int)` to `State Int x`

Introducing monad transformers.

Non-isomorphic representations.

Reactive extensions.

Going to OTP, distributed, replicated, supervised ...

“Can I apply this to GHC Haskell?”

“How do I refactor my Erlang + JavaScript?”

“Who cares about text files these days?”

“How do I parallelise this code?”

Questions?

www.cs.kent.ac.uk/projects/wrangler

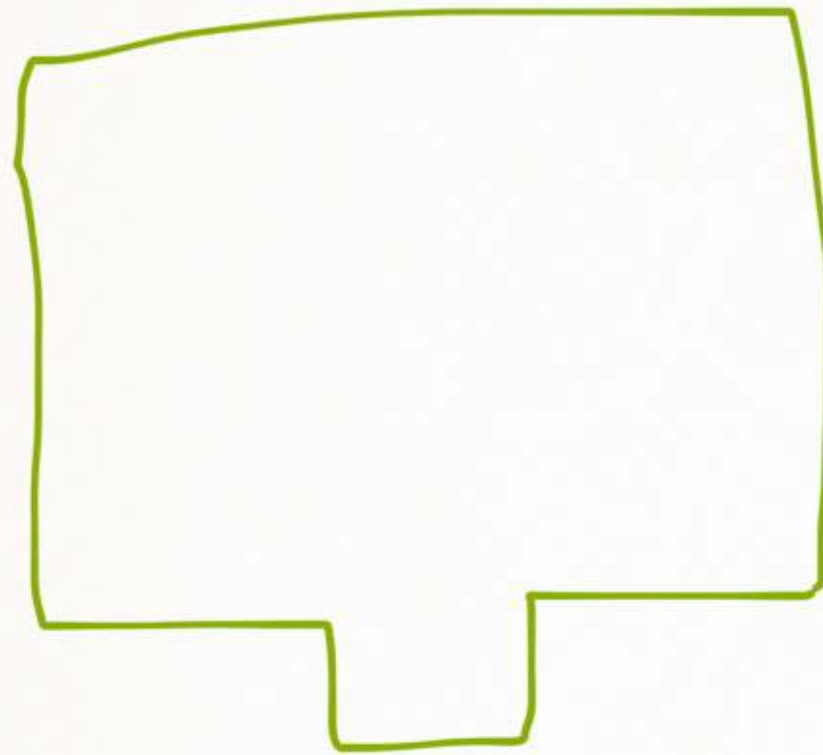


API migration

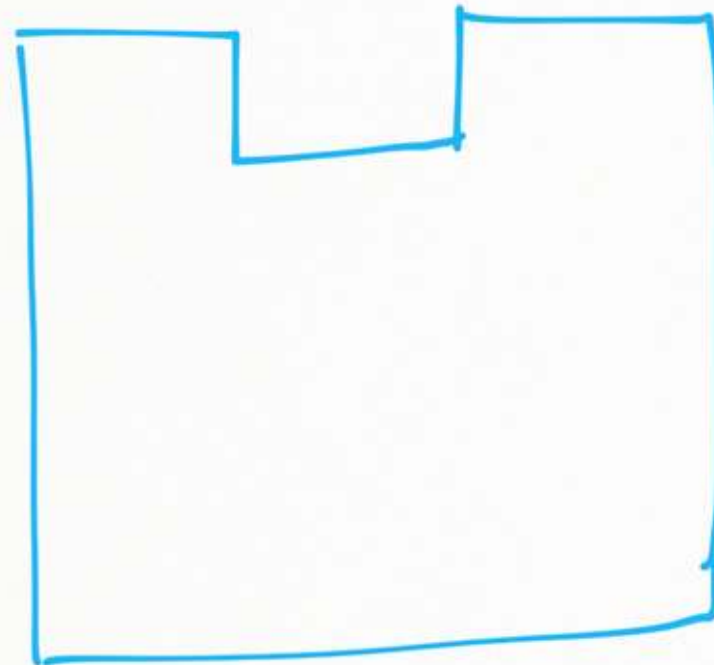
Scenario: system upgrade accompanied with a change in API.

Example from Erlang standard distribution: the regular expression library from `rexp` to `re`.

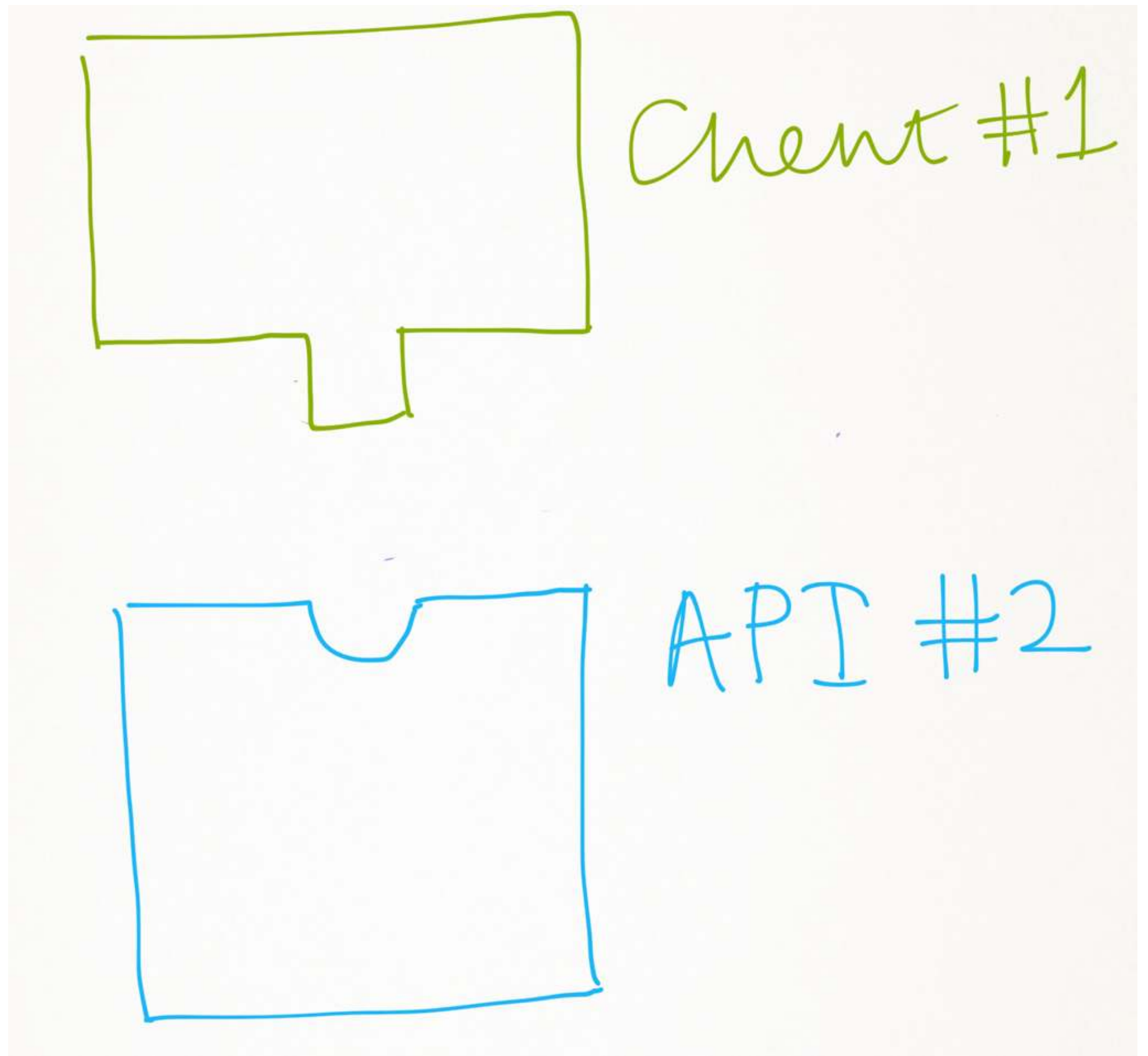
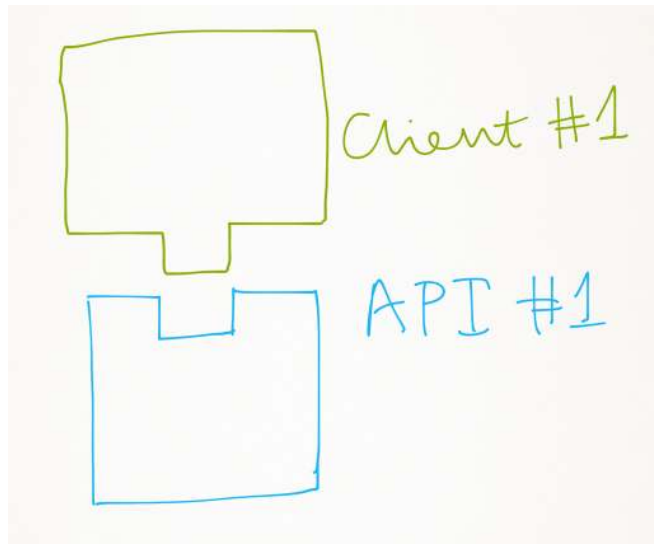
How to refactor client code to accommodate this?

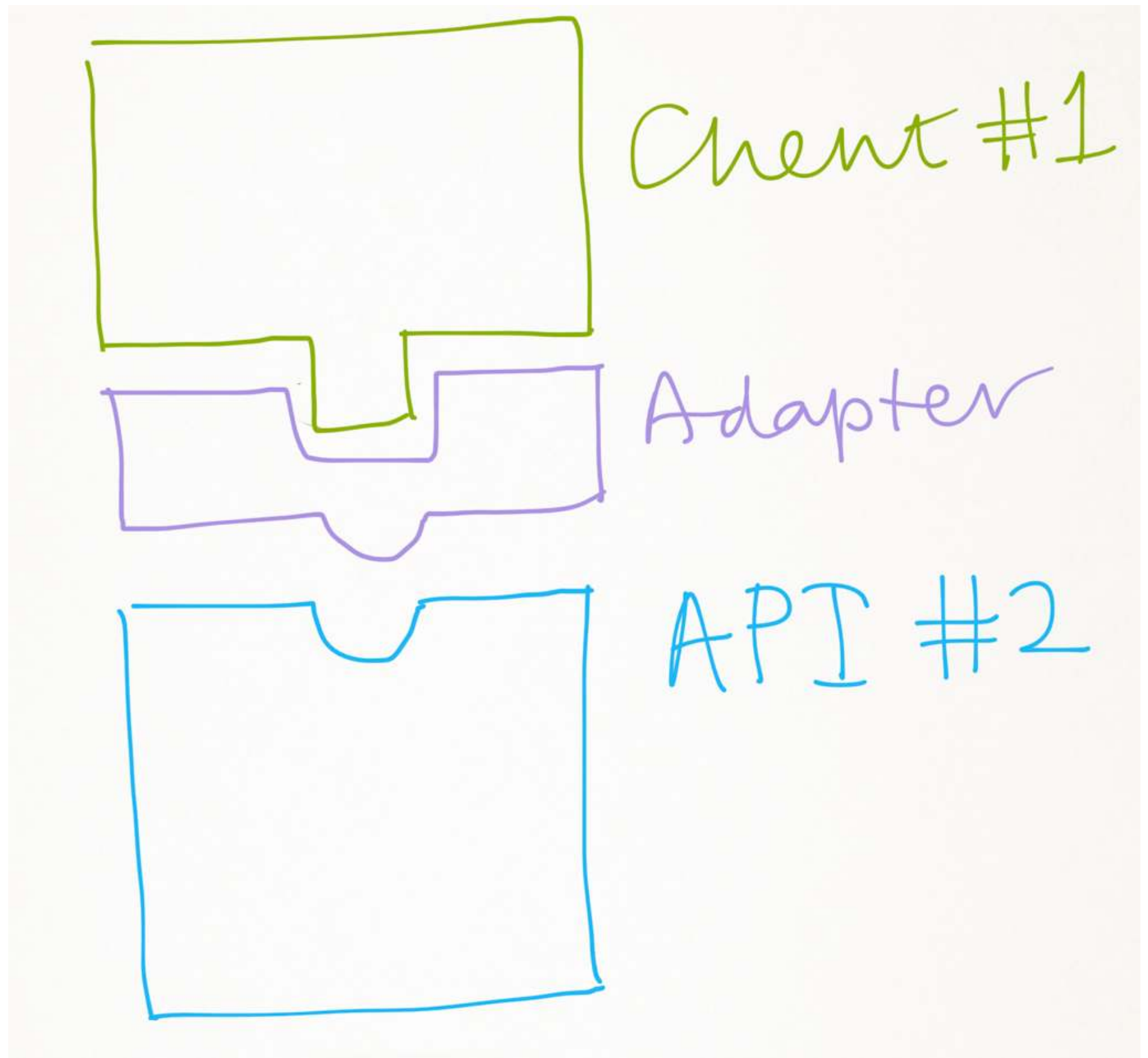
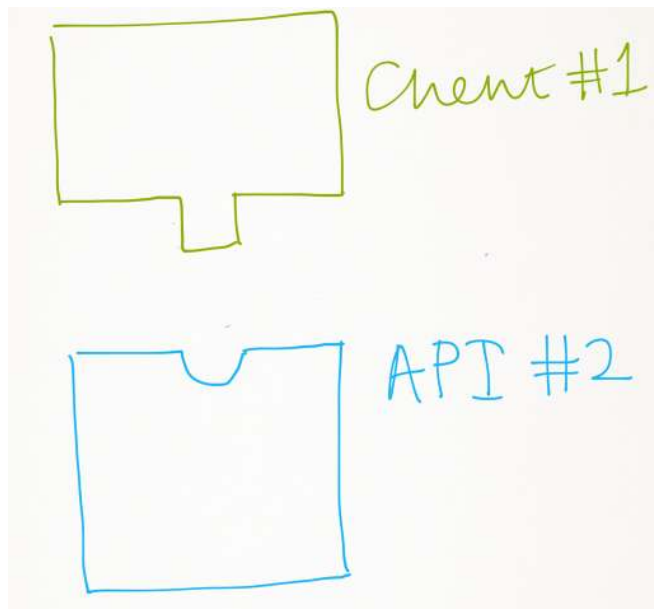
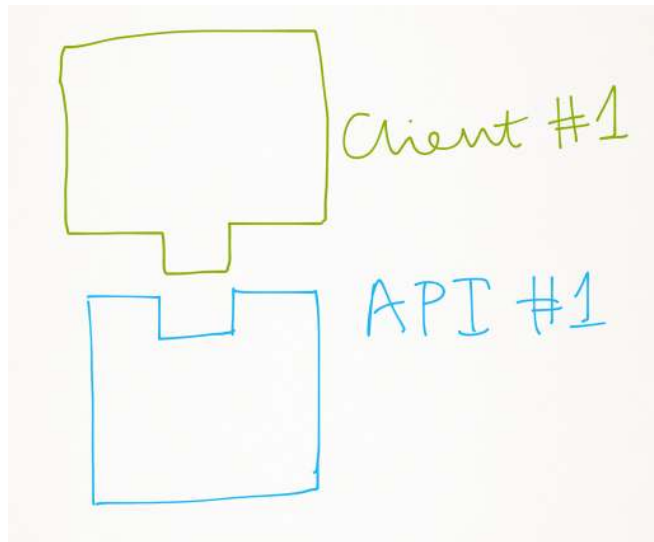


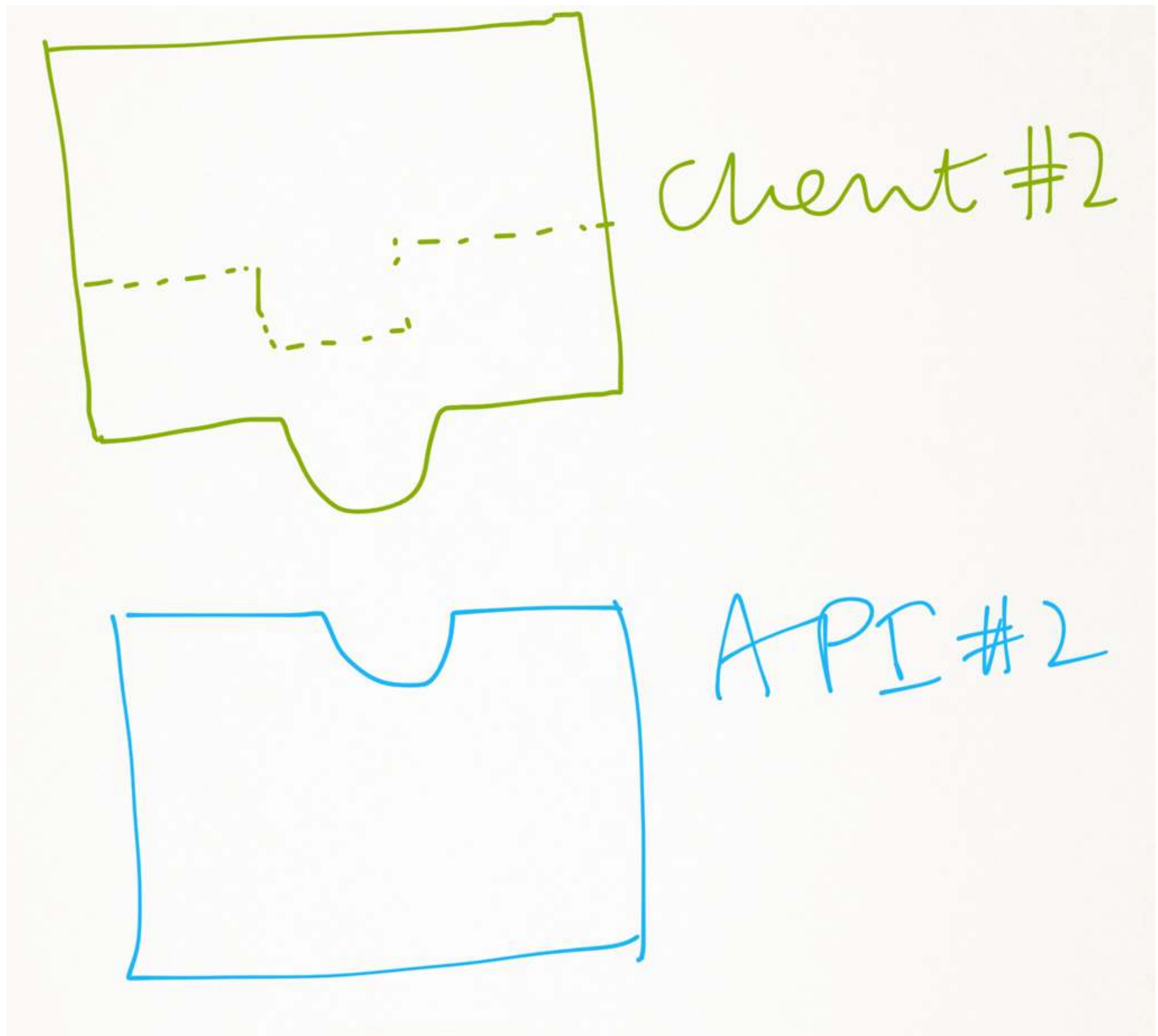
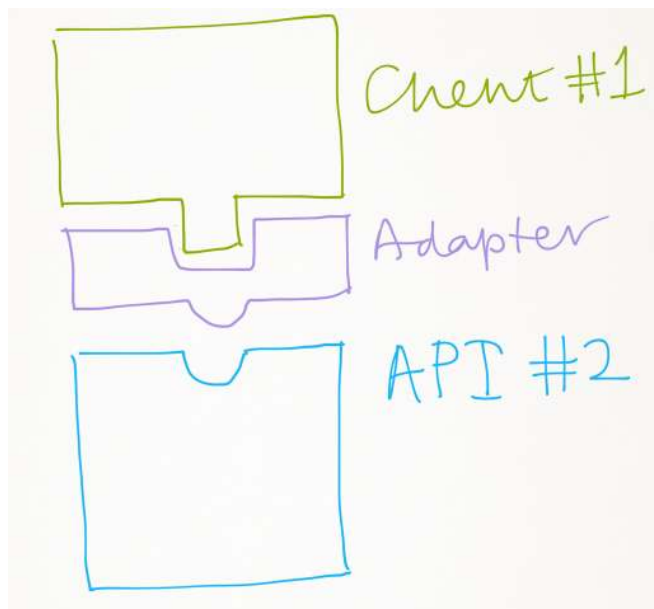
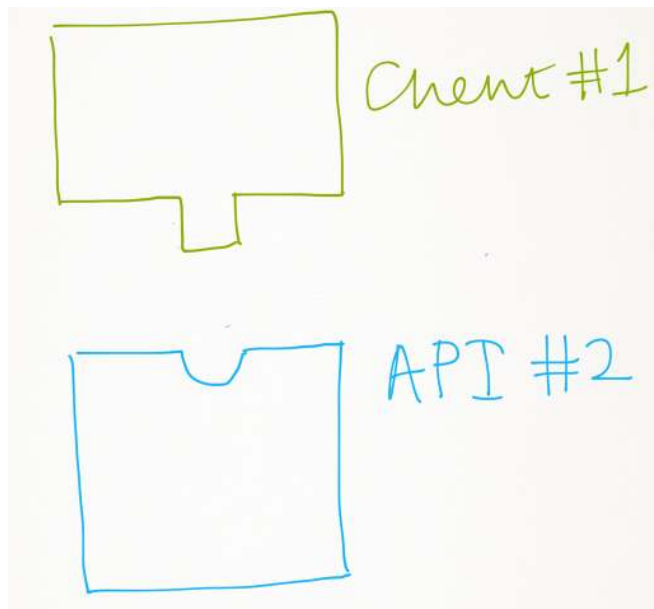
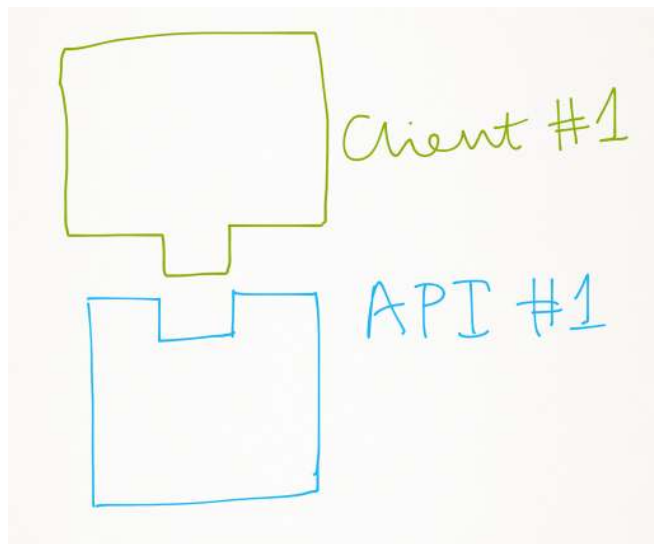
Client #1



API #1







Adapter regexp to re

```
match(String, RegExp) ->
  try re:run(String, RegExp, [global]) of
    {match, Match} ->
      {Start0, Len} = lists:last(
                                lists:ukeysort(
                                    2, lists:append(Match))),
      Start = Start0+1,
      {match, Start, Len};
  nomatch -> nomatch
catch
  error:_->
    {error, Error}=re:compile(RegExp),
    {error, Error}
end.
```


Client #1

```
secret_path(Path, [[NewDir] | Rest], Dir) ->
  case regexp:match(Path, NewDir) of
    {match, _Start, _Len} when Dir == to_be_found ->
      secret_path(Path, Rest, NewDir);
    {match, _Start, _Len} ->
      secret_path(Path, Rest, Dir);
    nomatch ->
      secret_path(Path, Rest, Dir)
  end.
```


Client #2

```
secret_path(Path, [[NewDir] | Rest], Dir) ->  
  case re:run(Path, NewDir, [global]) of  
    {match, _Match} when Dir == to_be_found ->  
      secret_path(Path, Rest, NewDir);  
    {match, _Match} ->  
      secret_path(Path, Rest, Dir);  
    nomatch ->  
      secret_path(Path, Rest, Dir)  
  end.
```

Client #1

```
document_name(Path) ->  
  case regexp:match(Path,"[^/]*\$") of  
    {match, Start, Len} ->  
      string:substr(Path, Start, Len);  
    nomatch -> "(none)"  
  end.
```

Client #2

```
document_name(Path) ->  
  case re:run(Path, "[^/]*\\$", [global]) of  
    {match, Match} ->  
      {Start0, Len}=lists:last(lists:ukeysort(2,Match)),  
      Start = Start0 + 1,  
      string:substr(Path, Start, Len);  
    nomatch -> "(none)"  
  end.
```